

17-214/514: Agentic Software Development

Course Introduction and Modern Software Engineering

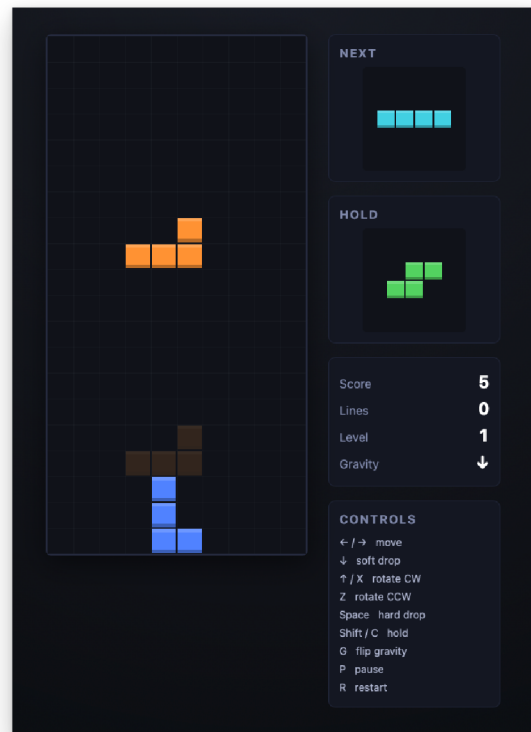
Hammad Ahmad and Claire Le Goues • Monday, August 24, 2026 • **Lecture**

Vibe coding FTW!

Last night I asked Claude: "Build me a playable Tetris game in the browser."

And here it is. It works.

So if **that** works... what's left to teach you?



Some of you enrolled in a different course

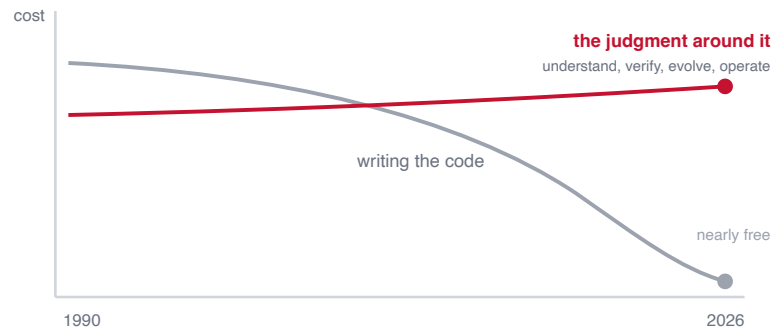
- The course catalog used to call this "Principles of Software Construction."
 - An even older course name had "...: Objects, Design, and Concurrency."
- It now says "Agentic Software Development."
- (Same course number. Yes, that is apparently allowed.)

Objects and concurrency are still in here. There's plenty of new material too. **Design stays front and center.**

Why are we here?

How do you engineer maintainable, verifiable software when writing the code is the easy part?

- Writing code used to be the slow, expensive part of the job.
- These days, an agent will hand you a 1000-line change in seconds.
- So we'll spend a semester on the parts that are still hard. (Spoiler: there are a lot of them.)



(An entirely made-up graph)

The shift

What changed in software engineering?

The old assumption

Writing correct code is
expensive.

Optimize for: clever algorithms,
careful manual implementation,
getting it written at all.

The new assumption

Writing code is **cheap**.
Understanding, verifying,
evolving, and operating it stays
expensive.

Optimize for: judgment, structure,
verification, supervision.

A few questions

- How many lines of code behind a service like Slack?
 - Millions. Maybe even tens of millions.
- How many lines to build a credible clone of the core?
 - About 10K. (People have done it over a weekend, **pre-AI**.)
- How many of those lines do **you** write by hand today?
 - Fewer every year. (And AI models are getting better.)

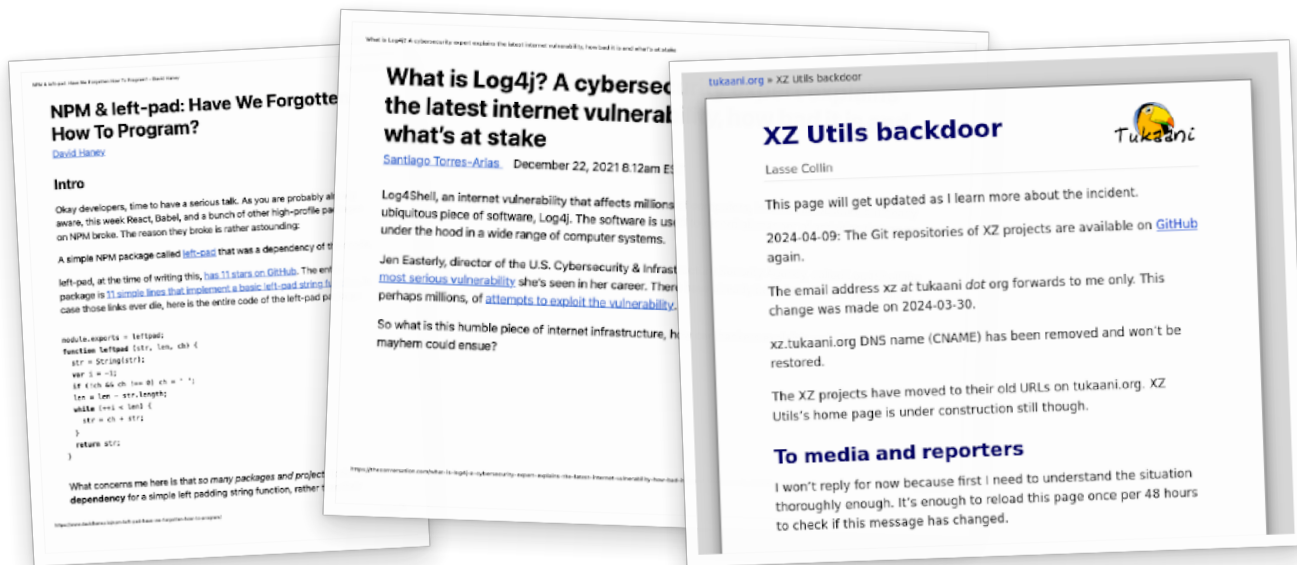


You'll see why we are mentioning Slack in a bit.

Nobody writes a million lines of code (by hand)

- Historically: you don't write a social network from scratch. You compose libraries.
 - `import android.*` => +12M lines you didn't write (and will never read).
- And your libraries use libraries. Which use libraries... You see where this is going?
- These days: you'll ask your agent to compose libraries, or **write new ones**.
 - But, a popular library has many users testing and debugging it. The code an agent just generated for you does not.

A word on using someone else's code



How much code do agents write today?



So, what's actually still hard?

Understanding

Why is it shaped like this? How does it fit into the org's other systems, and which customers rely on it?

Verifying

How do we know it's correct? What does correct mean, even?

Evolving

Can we change it next year without the whole thing falling apart? What breaks, and who notices, when we do?

Operating

Can we deploy it, watch it, and keep it alive when a server is down at 3am? Or when millions of simultaneous requests come in unexpectedly?

Ownership and accountability matter, regardless of who wrote the code!

Programming != software engineering

- Programming is making it work. (On your machine, and quickly.)
 - Agents are great at this!
- Software engineering is keeping it working over time (years of change), at scale (millions of lines, millions of users), with other people (teammates, successors, and now agents), and with dependents (other services).
 - Agents can help *a lot* with this, but they need your supervision.

We want to teach you software **design**. You will design software for change, scale, and operations.

This is **not** a course about...

- How to use a particular AI tool
- How to build AI agents or frameworks
- Prompt engineering
- "Vibe coding"

...so what **is** it about?

**Designing systems you
can trust, evolve, and run,
when the code is cheap
and quick to write.**

Five questions we'll visit this semester

WEEKS 1-2

Can we trust it?

Design for verification:
workflows, testing, specs,
coverage.

WEEKS 3-7

Can it change?

Design for change: modularity,
coupling, refactoring, drift.

WEEKS 6-7

Can it be reused?

Design for reuse: APIs, libraries,
frameworks, patterns.

WEEKS 8-11

Can it grow?

Design for scale: architecture,
data models, concurrency.

WEEKS 11-14

Can we run it?

Design for operations: cloud,
DevOps, observability, security.

FIRST

Verify, then design

Generated code is cheap.
Verification will require your
time and attention.

(For old-title loyalists: objects kind of never left, design is the entire map above, and concurrency is week 10.)

Working with coding agents

So... just ask the agent to do it?

- Sometimes: yes! You're done in seconds.
- Sometimes: it hands you something plausible, confident, and wrong.
 - You'll see what we mean when you work on Assignment 1.
- We'll practice knowing which of these situations you're in **before** you ship your code.

What are agents good at (currently...)?

- Understanding and explaining a (local) project you haven't seen before, or a snippet of code you don't understand.
- Well-scoped, clearly specced-out local changes.
- Rapid whole-system prototypes.
- API knowledge.
- Iterating to (and decomposing and executing) an acceptable design.
- Documenting. (This can be a double-edged sword.)
- Boosting your productivity*.
- Confidently making incorrect claims. (And gaslighting you in the process. Try to avoid cognitive surrender.)
- Overengineering.
- A bunch of other stuff.

...and what are they bad at (again, currently...)?

- Consistent global structure (a coherent picture across a whole organization).
- Testing sufficiently and *effectively*.
- Remembering why a legacy system was built a certain way. (Unless there is good documentation.)
- Knowing the implications of a change across the board.
- Maintaining context.
- Reading people. Or doing what you thought you said, not what you actually said.
- Estimating time and effort.
- Finishing up a task without going down rabbit holes, unless you have a good "definition of done".
- *Many more things!* We'll see them this semester.

(At the risk of sounding overly dramatic...)

— **The part you need to pay most attention to (the "dangerous part") is what the agent can **almost** do.** —

Should you let the agent write it?

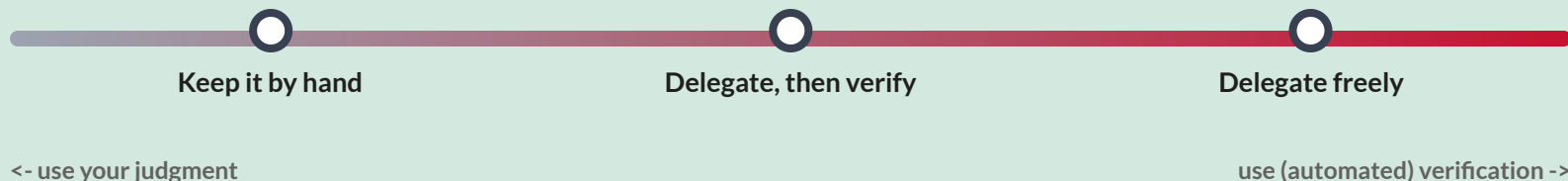
It depends.

... on what?

- **Reach.** How far does a wrong answer spread?
- **Verifiability.** Can you cheaply check the result, or are you just vibing and hoping?
- **Reversibility.** How bad is it if it's wrong and nobody notices for weeks?

"it depends" = reading these three dials

Would you ship it?



- Rename a symbol across 40 files
- Choose the data model for a new feature
- Add input validation to an API endpoint
- Write the locking for a shared queue
- Generate unit tests for a function
- Decide the boundary between two subsystems

How much design does this need?

- **Light:** Ask, smell check the plan, check the code that comes back.
- **Heavy:** Plan out, interrogate, and iterate on the design first.
 - ↳ then decompose into agent-sized tasks, with verification.

Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025

Ruanqianqian (Lisa) Huang^{*†}
lisahuang@cornell.edu
Cornell University
USA

Avery Reyna^{*}
avery.reyna16@gmail.com
Independent
USA

Sorin Lerner
sorin.lerner@cornell.edu
Cornell University
USA

Haijun Xia
haijunxia@ucsd.edu
UC San Diego
USA

Brian Hempel
bhempel@ucsd.edu
UC San Diego
USA

Huang et al., [arXiv:2512.14012](https://arxiv.org/abs/2512.14012) (Cornell and UC San Diego). Observations with 13 professional developers, plus a survey of 99.

Professional software developers don't vibe, they control.

"The agent wrote it" is not a defense

- Even if the agent generated it, **you** are accountable for it.
 - cf. Google's 2025 DORA report.
- Correctness, security, and license compliance stay yours.
- Let's avoid shipping the wrong thing faster!

AI, the great amplifier

As we established from the [2024 report](#) as well as the special report published this year called "[Impact of Generative AI in Software Development](#)", organizations are continuing to heavily adopt AI and receive substantial benefits across important outcomes. And there is evidence of learning to better integrate these tools into our workflow. Unlike last year, we observe a positive relationship between AI adoption on both software delivery throughput and product performance. It appears that people, teams, and tools are learning where, when, and how AI is most useful. However, [AI adoption does continue to have a negative relationship with software delivery stability](#).

This confirms our central theory - AI accelerates software development, but that acceleration can expose weaknesses downstream. Without robust control systems, like strong automated testing, mature version control practices, and fast feedback loops, an increase in change volume leads to instability. Teams working in loosely coupled architectures with fast feedback loops see gains, while those constrained by tightly coupled systems and slow processes see little or no benefit.

Key findings from the 2025 report

Beyond this central theme, this year's research highlighted the following about modern

Google DORA, State of AI-assisted Software Development 2025

Why verification comes first

Generate is cheap. Verify is the bottleneck.



The idea: make software easy to evaluate and trust.

Why? Because verification takes time. And therefore money.

What does "design for verification" buy us?

Software you can actually check is software that is:

Testable

You can pin behavior down with examples.

Observable

You can see what it's doing while it runs.

Reviewable

Changes arrive small enough to actually read.

Modular

A wrong answer stays in one place instead of everywhere.

We'll start with verification. Everything else stacks on top of it.

How this course works

(If it wasn't clear already...)

— **You will work with agents. A lot.** —

Assignments

A1

Repair

A vibe-coded system. Diagnose, fix, verify.

A2

Build

Slack v0.1. Design and implement.

A3

Evolve

New requirements on top of your old decisions.

A4

Refactor

A big system you did not design. Critique it, change it safely.

A5

Scale

Make your system survive growth.

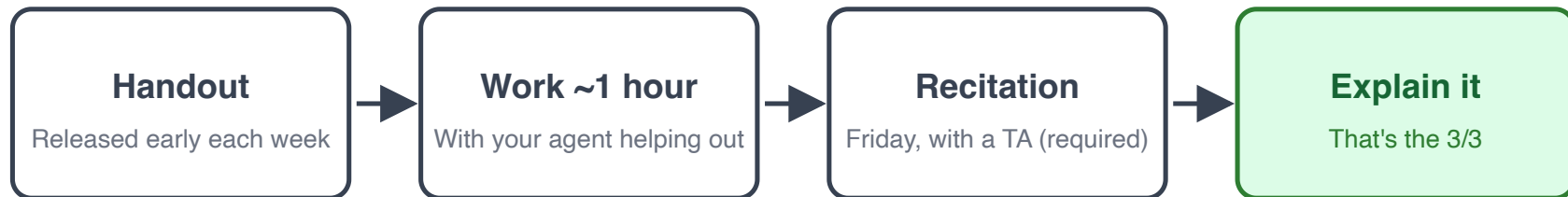
A6

Operate

Deploy your system, watch it, keep it alive.

Labs

- 13 labs, one per week
- Three concrete milestones each. Show all three at recitation: 3/3.
- Graded for completeness, not correctness.
 - You explain what happened and why, out loud, to a TA. (Your agent is not invited to that conversation.)



What we grade

- We primarily focus on grading your decisions, and the reasoning behind them.
- Things we want you to care about:
 - Verification rationale
 - Design choices
 - Decision records
 - Restraint
 - Agent supervision

A typical rubric breakdown

Dimension	What we look at
Functionality	Passes the acceptance bar. (Practice restraint!)
Verification	Your tests and the rationale: what's covered, what deliberately isn't, why.
Design & decomposition	Boundaries, data model, structure that fits the problem.
Engineering memory	Decision records (including alternatives, rationale, updates, etc.).
Supervision & process	How you directed the agent: plan, log, reflection, and a short spoken defense.
Assignment theme	Evolution, behavior preservation, scale, operations.

Your tooling/stack for the semester

You'll get comfortable with

Coding agents (Claude Code, Codex, Cursor, ...)

Agents will write most, if not all, of the code for you. You'll supervise.

IDEs & editors

For you to read, navigate, and review code.

Build systems and CI

Automated gates that run on every change. (Does it still compile? Do the tests still pass?)

Version control

Small, reviewable commits. Backups in case things go wrong.

And more... (cloud and deployment, containers, monitoring, etc., as the course goes on)

"What language are we going to be using?"

Whatever makes sense for the problem. This is not a Java/TypeScript/Python/<insert-language-here> course.

- Syntax and boilerplate are the low-level layer. If you don't know a language, ask your agent to write and/or explain the code.
- You should focus on the layer that's still hard. Is this the right design, is it actually correct, will it survive change, will it scale, etc.?
- **Program comprehension**, not code comprehension.

What we expect of you

How we'll work this semester

- **Design** a good solution, then decompose it.
 - AI can help with both!
- **Verify** generated output. Don't rely on vibes.
- Commit frequently, at meaningful boundaries. Treat your version control history as a review log.
- Disclose how you used AI tools.
 - You'll find specific instructions in assignment handouts.

You can use AI tools freely

But:

- **Own and understand** everything you submit. You must be able to explain every key decision.
- Again, "Claude said so" is not an acceptable defense in this course (and beyond, for what it is worth).

Course staff

Instructors



Hammad Ahmad

hammad@andrew.cmu.edu. TCS 341.



Claire Le Goues

clegoues@cs.cmu.edu. TCS 363.

Teaching assistants

Harrison Green

Alex Feies

**Alex Torres
Vivaldo**

David Hu

Course logistics

- **Lectures:** M/W, 11:00 a.m. to 12:20 p.m., Tepper (TEP) 1403.
- **Recitations:** Fridays, in your section. This is where labs get signed off, so attendance is required.
 - Please only go to the recitation section you're enrolled in.
- **Course site:** cmu-17-214.github.io/f2026 (for the syllabus, schedule, policies, and the course calendar).
- We will also use Canvas, Gradescope, and Piazza.

Grading and policies

- **Assignments:** 50%. Six of them, equally weighted, each with a short spoken defense. Assignments 2-6 also have checkpoints.
- **Exams:** two midterms at 7.5% each, plus a 15% final.
- **Labs:** 10%, graded for completeness at recitation. Two lowest dropped.
- **In-class quizzes:** 10%, graded for completeness. Two lowest dropped.
- **Late days:** 5 free across the semester on final assignment deadlines, then 10% off per day. Checkpoints and Assignment 6 are not eligible.
- You can find the full agent-use, integrity, and deadline policies on the course site.

Attendance and participation

- **Lectures:** Attendance strongly recommended. We will frequently have short in-class quizzes and activities.
 - We do not accommodate remote attendance.
 - Lectures are being recorded. You can find the recordings at Canvas -> Panopto Recordings.
- **Recitations:** Attendance required. (See the Labs slide.)
- **Devices:** Off during lecture, please. (Exception: the Canvas quiz.)

"What AI tool should I use?"

- **Claude Code** is the officially supported tool.
- Any other tool is fine (Cursor, Codex, Gemini CLI, Aider, ...), with one condition. It must export a complete transcript.
 - Making that export work is your/your agent's job. (See Lab 1 for details.)
- You are responsible for the costs. Budget about \$100/month (e.g., Claude Max 5x). Cheaper and free options exist (e.g., Copilot Student Pack, Gemini free tier), but you may hit token limits.
- Get a tool working **this week**. You will need it for Lab 1.

Before we leave...

We need this to get you all set up for homework assignments. Make sure you are signed in using your CMU account.

forms.gle/6M5gQ7s7X5sPxVgDA



Summary

- Code is cheap and abundant. Trust, change, reuse, scale, and operations remain the hard parts.
- We will learn **software design with agents**.
- Agents are really good at many things, but they are not replacing you.
- You can use agents freely, but you are responsible for everything you submit.
- (And, the answer to "would you do A or B?" is almost always "it depends.")

What's next

W1 A1 out	W2	W3 A1 due	W4	W5 A2 due · M1	W6	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
---------------------	-----------	---------------------	-----------	--------------------------	-----------	---------------------	-----------	---------------------	------------------	------------	----------------------	------------	----------------------

- **Wednesday:** how to actually work with a coding agent, and getting started with design for verification.
- **Friday:** Lab 1: Course infrastructure, and your first supervised agent task.
- On the horizon: Assignment 1. You will have fun fixing a vibe-coded system.